Proyecto Ciber Guardianes

Gestor de Control y Seguridad en Juegos Infantiles

Fundamentación del Dominio y Adaptación del Modelo

En la actualidad, los niños interactúan activamente en plataformas de juego en línea (como *Roblox, Minecraft o Brawl Stars*), exponiéndose a riesgos severos como contacto no supervisado con desconocidos, microtransacciones impulsivas no autorizadas y contenidos inadecuados para su etapa de desarrollo. Los padres, tutores y educadores requieren una herramienta informática centralizada para catalogar, evaluar y administrar los niveles de riesgo y permisos de acceso de cada videojuego.

Para resolver este problema práctico, se extendió la entidad genérica **Registro** provista en la plantilla inicial del laboratorio, transformándola en la entidad de dominio **JuegoSeguro**. Esta adaptación no se limita a un mero cambio cosmético o de etiquetas, sino que incorpora atributos específicos de ciberseguridad y reglas de evaluación dinámica.

Atributo Base	Atributo Adaptado	Tipo Java	Descripción / Regla de Negocio
id	id	String	Identificador único intransferible (ej: 'GAME-001').
nombre	titulo	String	Nombre comercial del videojuego (ej: 'Roblox'). Obligatorio.
categoria	plataforma	Enum	Plataforma principal: MOBILE, PC, CONSOLA, WEB.
estado	nivelRiesgo	Enum	Clasificación de riesgo: SEGURO, PRECAUCION, BLOQUEADO.
(Nuevo)	edadMinima	int	Edad mínima recomendada por PEGI/ESRB (rango 0 a 18 años).
(Nuevo)	tieneChatAbierto	boolean	Indica si permite comunicación abierta con extraños.

Atributo Base	Atributo Adaptado	Tipo Java	Descripción / Regla de Negocio
(Nuevo)	comprasIntegradas	boolean	Indica si incluye micropagos o cajas de botín (loot boxes).

Componentes Gráficos Swing (Cumplimiento de 8 Tipos)

Para dar estricto cumplimiento al requisito del laboratorio de contar con al menos **8 tipos de componentes Swing con funcionalidad real** (5 base + 3 nuevos), la interfaz gráfica integra los siguientes elementos:

Tipo de Componente Swing	Categoría	Función Real en Ciber Guardianes
JTextField	Base (1/5)	Ingreso de identificador ('id') y título del juego.
JComboBox	Base (2/5)	Selección de la plataforma desde el Enum (MOBILE, PC, CONSOLA, WEB).
JCheckBox	Base (3/5)	Marcación de opciones lógicas ('tieneChatAbierto' y 'comprasIntegradas').
JButton	Base (4/5)	Disparo de acciones CRUD (Guardar, Modificar, Eliminar, Limpiar, Buscar).
JTable	Base (5/5)	Visualización tabular del catálogo mediante JuegoTableModel.
JSpinner	**Nuevo (1/3)**	Selección numérica con control de rango estricto (0 a 18) para 'edadMinima'.
JRadioButton + ButtonGroup	**Nuevo (2/3)**	Filtro rápido excluyente de visualización por NivelRiesgo (Todos, Seguro, Precaución, Bloqueado).
JTextArea + JScrollPane	**Nuevo (3/3)**	Despliegue y redacción de observaciones de seguridad y justificación del riesgo asignado.

Reglas de Negocio y Validaciones de Seguridad

El sistema implementa 4 capas de validación y reglas de negocio para garantizar la coherencia de los datos y proteger las configuraciones de seguridad infantil:

- **Validación de Campos Obligatorios y Rangos:** El título del videojuego no puede estar vacío ni compuesto únicamente por espacios. La edad mínima debe encontrarse obligatoriamente dentro del rango de 0 a 18 años.
- **Evaluación Automática de Riesgo (Regla Propia):** Si un videojuego combina chat abierto sin filtros (tieneChatAbierto == true) y compras de micropagos (comprasIntegradas == true), el modelo impide automáticamente otorgarle el nivel de riesgo SEGURO, forzándolo a PRECAUCION o BLOQUEADO.
- **Protección de Borrado de Juegos Bloqueados (Regla Propia):** Se prohíbe la eliminación directa mediante la Fachada de cualquier videojuego catalogado en estado BLOQUEADO, evitando que un menor de edad elimine restricciones de la aplicación.
- **Unicidad de Identificador (Control de Duplicados):** Antes de registrar un nuevo juego, la Fachada consulta al repositorio JSON para verificar que el 'id' no se encuentre registrado previamente, arrojando una excepción clara si existe colisión.

Arquitectura del Sistema (MVC + Fachada en Español)

Atendiendo a las especificaciones obligatorias de la cátedra, la estructura de paquetes se ha estandarizado completamente en **idioma español**, desglosando las responsabilidades de la siguiente manera:

```
src/
└── ar.edu.ciberseguridad.juegos/
    ├── app/
    │   └── Main.java                    # Inicialización y arranque
    ├── modelo/
    │   ├── JuegoSeguro.java             # Entidad principal con encapsulamiento
    │   ├── Plataforma.java              # Enum (MOBILE, PC, CONSOLA, WEB)
    │   └── NivelRiesgo.java             # Enum (SEGURO, PRECAUCION, BLOQUEADO)
    ├── persistencia/
    │   └── JuegoJsonRepository.java     # Lectura y escritura de datos JSON
    ├── fachada/
    │   └── CiberSeguridadFacade.java    # Aislamiento de lógica y persistencia
    ├── controlador/
    │   └── JuegoController.java         # Manejador de eventos Swing
    └── vista/
        ├── VentanaPrincipal.java        # Formulario GUI Swing
```

```
├── FormularioJuegoDialog.java       # Diálogo emergente
└── JuegoTableModel.java             # Adaptador de JTable
```

A. Entidad de Dominio (modelo/JuegoSeguro.java)

```
package ar.edu.ciberseguridad.juegos.modelo;

public class JuegoSeguro {
    private String id;
    private String titulo;
    private Plataforma plataforma;
    private NivelRiesgo nivelRiesgo;
    private int edadMinima;
    private boolean tieneChatAbierto;
    private boolean comprasIntegradas;

    public JuegoSeguro(String id, String titulo, Plataforma plataforma, NivelRiesgo
nivelRiesgo,
                       int edadMinima, boolean tieneChatAbierto, boolean comprasIntegradas) {
        if (titulo == null || titulo.isBlank()) {
            throw new IllegalArgumentException("El título no puede estar vacío.");
        }
        if (edadMinima < 0 || edadMinima > 18) {
            throw new IllegalArgumentException("La edad mínima debe estar entre 0 y 18
años.");
        }
        this.id = id;
        this.titulo = titulo;
        this.plataforma = plataforma;
        this.edadMinima = edadMinima;
        this.tieneChatAbierto = tieneChatAbierto;
        this.comprasIntegradas = comprasIntegradas;
        this.nivelRiesgo = evaluarRiesgo(nivelRiesgo);
    }

    private NivelRiesgo evaluarRiesgo(NivelRiesgo sugerido) {
        if (tieneChatAbierto && comprasIntegradas && sugerido == NivelRiesgo.SEGURO) {
            return NivelRiesgo.PRECAUCION; // Regla de negocio automática
        }
        return sugerido;
    }

    // Getters y Setters encapsulados...
```

```
    public String getId() { return id; }
    public String getTitulo() { return titulo; }
    public Plataforma getPlataforma() { return plataforma; }
    public NivelRiesgo getNivelRiesgo() { return nivelRiesgo; }
    public int getEdadMinima() { return edadMinima; }
    public boolean isTieneChatAbierto() { return tieneChatAbierto; }
    public boolean isComprasIntegradas() { return comprasIntegradas; }
}
```

B. Fachada de Operaciones (fachada/CiberSeguridadFacade.java)

```
package ar.edu.ciberseguridad.juegos.fachada;

import ar.edu.ciberseguridad.juegos.modelo.*;
import ar.edu.ciberseguridad.juegos.persistencia.JuegoJsonRepository;
import java.util.List;

public class CiberSeguridadFacade {
    private final JuegoJsonRepository repository;

    public CiberSeguridadFacade(JuegoJsonRepository repository) {
        this.repository = repository;
    }

    public List<JuegoSeguro> obtenerTodosLosJuegos() {
        return repository.cargarDesdeJson();
    }

    public void registrarJuego(JuegoSeguro juego) throws Exception {
        if (repository.existePorId(juego.getId())) {
            throw new Exception("Ya existe un juego registrado con el ID: " + juego.getId());
        }
        repository.guardar(juego);
    }

    public void eliminarJuego(String id) throws Exception {
        JuegoSeguro juego = repository.buscarPorId(id);
        if (juego != null && juego.getNivelRiesgo() == NivelRiesgo.BLOQUEADO) {
            throw new Exception("No se puede eliminar un juego BLOQUEADO por protección
de seguridad.");
        }
        repository.eliminar(id);
    }
}
```

Persistencia JSON y Manejo de Errores

La capa de persistencia (persistencia/JuegoJsonRepository.java) asegura que las operaciones de Alta, Baja y Modificación perduren entre distintas ejecuciones de la aplicación. Cumpliendo con los requerimientos técnicos del laboratorio:

- **Inicialización sin Archivo Previo:** Si la aplicación se ejecuta por primera vez en un entorno limpio sin el archivo 'juegos.json', el sistema captura la ausencia del recurso e inicializa automáticamente una colección vacía sin lanzar excepciones no controladas ni interrumpir la GUI.
- **Tolerancia a Errores y Recuperación:** Ante eventuales fallos de i/o o corrupción de datos, el repositorio notifica la eventualidad mediante JOptionPane a través del controlador sin cerrar la aplicación inesperadamente.

El proyecto Ciber Guardianes demuestra una resolución práctica y robusta al problema de la supervisión parental en los videojuegos infantiles en línea. A través de la redefinición del modelo base en la entidad JuegoSeguro, la aplicación trasciende una simple gestión de datos para convertirse en un sistema de evaluación activa que protege la experiencia digital de los menores.

Puntos Clave del Desarrollo:

- **Arquitectura Decoplada:** La implementación del patrón MVC junto con una Fachada (CiberSeguridadFacade) garantiza una estricta separación de responsabilidades. La interfaz gráfica Swing no realiza operaciones directas de persistencia ni decide reglas de negocio.

- Lógica de Negocio Automática: El sistema aplica criterios de seguridad en tiempo real, evaluando automáticamente como PRECAUCION o BLOQUEADO aquellos juegos con chat abierto y compras integradas, además de proteger el borrado no autorizado de registros bloqueados.

- Interfaz Rica y Persistencia: La incorporación de componentes avanzados (JSpinner, JRadioButton, JTextArea, JTable) junto a la persistencia local en archivos JSON asegura una experiencia de usuario intuitiva, robusta e independiente de servicios externos.

En conjunto, la solución no solo cumple con la totalidad de los requerimientos técnicos del laboratorio de Programación 2, sino que ofrece una herramienta funcional, extensible y con impacto real en el ámbito de la ciberseguridad familiar.

Bibliografia didáctica y disciplinar:

- Entertainment Software Rating Board. (s. f.). Video game ratings, categories, and parental controls. ESRB. [suspicious link removed]
- Equipo CiberGuardianes. (2026). Proyecto CiberGuardianes: Gestor de control y seguridad en juegos infantiles [Documento técnico no publicado].
- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Google. (2023). Gson user guide. GitHub. [suspicious link removed]
- Oracle Corporation. (2024). Java Platform, Standard Edition & Swing API documentation. Oracle Docs. [suspicious link removed]
- Pérez, D. (2026a). Programación 2: Laboratorio de Java [Consigna y guía de evaluación de laboratorio].
- Pérez, D. (2026b). Prototipo de inicio en Java Swing [Código fuente de software].

Página web:

- https//notebook.google.com.